Synthetic Financial Time Series Generation

GAN Benchmark: TimeGAN · QuantGAN · FinGAN

BRICS Emerging Market Indices

Collaboration	Universitat Politècnica de Catalunya (UPC)
Document type	Project Status & In-Depth Technical Review
Purpose	Paper publication preparation
Models	TimeGAN (RNN) · QuantGAN (TCN-WGAN-GP) · FinGAN (CNN-WGAN-GP)
Markets	BOVESPA · FTSE JSE · MSCI · NIFTY50 · SHANGHAI
Generated	2026-08-30

Project Status Overview

Research context · data pipeline · model status · open tasks

Research Goal

- Benchmark three GAN architectures for synthetic financial log-return generation
- Evaluate on 5 BRICS emerging-market indices (multi-market generalisation)
- Use literature-anchored hyperparameter floors (Gulrajani 2017); validate temporal robustness with walk-forward evaluation (CTBench protocol)
- Compare stylized-fact reproduction: fat tails, volatility clustering, long memory
- Prepare peer-reviewed paper in collaboration with UPC

Data Pipeline

- Source: 5 CSV files — BOVESPA, FTSE JSE, MSCI, NIFTY50, SHANGHAI
- Feature: daily log return $r_t = \ln(P_t/P_{t-1})$
- **No outlier clipping** applied to log returns (deliberate — see Code Audit)
- Temporal 80 / 10 / 10 split (no shuffle) — saved as Parquet files; train set used for GAN training
- GAN training: 15 models (3 GANs × 5 markets); each model trains on 128-step windows from its own market only — no cross-market data mixing
- Walk-forward evaluation: 5 rolling folds per market on that market's test series (CTBench protocol); model retrained each fold

TimeGAN

Yoon et al. (2019)[4], NeurIPS 32. 4-phase training (autoencoder → supervisor → joint GAN) using GRU networks. BCE loss + moment matching. MinMax $[0, 1]$ normalisation. **Current: epochs=5 (far too low — see Audit)**

QuantGAN

Wiese et al. (2020)[5], Quantitative Finance. WGAN-GP + TCN with causal dilated convolutions. Adam $\beta_1 = 0$, $\beta_2 = 0.9$, **n_critic=5** (Gulrajani[3] 2017 floor), $\lambda_{gp} = 10$.

FinGAN

Custom CNN-WGAN-GP. Generator: FC → 3× `ConvTranspose1d`. Critic: `Conv1d` + LayerNorm. seq_len must be divisible by 8. Same Adam/WGAN-GP settings as QuantGAN.

Pipeline Status

- **DONE:** Reproducibility seeds; deprecated API fixes; literature-anchored hyperparameter floors (n_critic=5, $\lambda_{gp} = 10$)
- **DONE:** `FinancialMetrics` rewritten with time-series-aware tests
- **DONE:** `n_critic` fixed at 5 for QuantGAN and FinGAN (Gulrajani[3] 2017 floor)
- **DONE:** Walk-forward validation (`walk_forward_evaluate()`, CTBench protocol, 5 rolling folds)
- **TODO:** Re-run pipeline to regenerate results with updated metric schema
- **TODO:** Increase TimeGAN epochs ($\geq$ 50–100 per phase)

The Problem — Financial Time Series from First Principles

Understanding the data before understanding the models

What is a Financial Time Series?

A financial time series is simply a sequence of numbers measured over time. For a stock or index: $P_1, P_2, \ldots, P_T$ where P_t is the closing price on day t.

Problem with raw prices: they grow over time (non-stationary), making comparisons across periods misleading. Statistical models require (approximate) stationarity.

Why Log Returns?

Instead of prices we compute the **log return** for each day:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln P_t - \ln P_{t-1}$$

Why the logarithm?

1. *Additivity:* $r_{1\to3} = r_{1\to2} + r_{2\to3}$ exactly (simple returns do not add).
2. *Approximation:* $\ln(P_t/P_{t-1}) \approx (P_t - P_{t-1})/P_{t-1}$ for small moves.
3. *Stationarity:* Log returns fluctuate around zero with no long-run trend.

The Core Problem: Returns Are NOT Normal

The simplest model: $r_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\mu, \sigma^2)$. This is the foundation of Black–Scholes and many classical models. **It is empirically wrong in the tails.**
Under $\mathcal{N}(0, \sigma^2)$ with $\sigma \approx 1.2\%$:

- A day with $|r| > 3\sigma \approx 3.6\%$ should occur $\approx 0.27\%$ of days
  $\Rightarrow$ roughly 1 day per year.
- Reality: 3–5× more frequent (**heavy tails**).
- A "5σ crash": under Gaussian $\approx$ once every **14 000 years**;
  in practice: several times per decade.

The 2008 crisis was described as a "25-sigma event" by some risk systems — probabilistically impossible under Gaussian assumptions.

What Are Stylized Facts?

Cont (2001)[7] coined the term **stylized facts**: statistical properties that appear consistently across virtually all financial markets and time periods. They are the fingerprint of financial returns.
Any generative model claiming to produce realistic synthetic returns MUST reproduce them.
The five most important:

1. **Heavy tails:** $P(|r| > x) \sim x^{-\alpha}$ (Pareto tail, $\alpha \approx 3$–5) $\Rightarrow$ kurtosis $\gg 3$.
2. **Near-zero autocorrelation:** $\text{Corr}(r_t, r_{t+k}) \approx 0$ for $k \geq 1$ (efficient market).
3. **Volatility clustering:** $\text{Corr}(|r_t|, |r_{t+k}|) > 0$ for $k = 1, \ldots, 100+$. Large moves beget large moves.
4. **Long memory:** ACF of $|r_t|$ decays as $k^{-\beta}$, $\beta \in (0, 1)$; Hurst exponent $H > 0.5$.
5. **Negative skewness:** crashes are sharper than rallies ($\text{skew}(r) < 0$ for equity indices).

Why Generate Synthetic Data?

If real data exists, why generate synthetic data?

- **Data augmentation:** deep learning models need large datasets; markets produce ∼250 observations/year.
- **Stress testing:** banks test risk models under extreme scenarios not in historical data.
- **Privacy:** real data may be proprietary; synthetic data with the same statistical properties can be shared freely.
- **Scenario generation:** sample many plausible futures for portfolio optimisation.

The challenge: a Gaussian generator fails immediately (wrong tails, no clustering). This motivates deep generative models.

The Solution Candidate: What is a Generative Adversarial Network?

From the counterfeiter–detective analogy to the minimax formulation (Goodfellow et al., 2014)

The Core Idea — A Two-Player Game

Goodfellow et al. (2014)[1] proposed training **two** neural networks that compete:

Generator G: takes random noise $z \sim p_z \to$ outputs fake sample $G(z) = \tilde{x}$.
Discriminator D: given a sample (real or fake) $\to$ outputs $D(x) \in [0, 1]$ (probability of being real).

Analogy: Counterfeiter (G) makes fake banknotes; detective (D) tries to spot them. Both improve iteratively until fakes are indistinguishable.

The **minimax objective**:

$$\min_G \max_D \; V(D,G) = \underbrace{\mathbb{E}_{x\sim p_{\text{data}}}\big[\log D(x)\big]}_{\text{reward } D \text{ for real}} + \underbrace{\mathbb{E}_{z\sim p_z}\big[\log(1 - D(G(z)))\big]}_{\text{penalise } G \text{ when caught}}$$

Reading each term:

- D *maximises*: wants $D(x_{\text{real}}) \to 1$ and $D(G(z)) \to 0$.
- G *minimises*: wants $D(G(z)) \to 1$ (fool D).

At the **Nash equilibrium** (Goodfellow et al., 2014, Proposition 1):

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} = \tfrac{1}{2} \;\forall x \quad \Longrightarrow \quad p_g = p_{\text{data}}$$

The generator has perfectly learnt the real distribution.

Critical Problem: Vanishing Gradients & Mode Collapse

Mode collapse: G learns to output only a few samples that reliably fool D, ignoring the true diversity of p_{data}.
Vanishing gradients: The GAN loss uses Jensen–Shannon (JS) divergence internally:

$$\text{JS}(p_{\text{data}} \| p_g) = \log 2 \quad \text{when } \text{supp}(p_{\text{data}}) \cap \text{supp}(p_g) = \varnothing$$

A *constant* has zero derivative with respect to G's parameters $\Rightarrow$ G receives no learning signal.
In practice: early in training, if G produces very different-looking samples from reality, D immediately says "fake" with no useful feedback on *how* G should improve.

What Makes GANs Suitable for Financial Returns?

Unlike parametric models (GARCH, EVT), a GAN:

- Makes **no distributional assumption** about the shape of returns; learns p_{data} implicitly.
- Can model arbitrarily complex joint distributions $p(r_1, r_2, \ldots, r_T)$, capturing both the marginal distribution *and* temporal dependencies.
- Scales to multivariate settings without specifying a copula.

The GAN sees sequences, not points. A window $[r_1, r_2, \ldots, r_{128}]$ is the training unit. D learns to recognise temporal patterns: bursts of high volatility, calm periods. G must generate sequences exhibiting those patterns.
This is what distinguishes GAN-based generators from models that generate each timestep independently (which would produce no volatility clustering even if the marginal distribution were correct).

Three Architectural Choices in This Benchmark

Model	Backbone	Key mechanism
TimeGAN	GRU (recurrent)	Latent embedding space
QuantGAN	TCN (causal dil.)	Long receptive field via dilation
FinGAN	CNN deconvolution	Noise $\to$ sequence via upsampling

A Better Distance Between Distributions

The JS divergence fails because it equals $\log 2$ (a constant) when supports do not overlap — giving G zero gradient.
Wasserstein-1 distance ("Earth Mover's distance"):

$$W(p,q) = \inf_{\gamma \in \Pi(p,q)} \mathbb{E}_{(x,y)\sim\gamma}\big[\|x-y\|\big]$$

where $\Pi(p,q)$ is the set of all joint distributions with marginals p and q.
Physical intuition: Think of p_{data} and p_g as two piles of sand. W is the minimum total work (mass × distance) needed to reshape one pile into the other.
Why is this better?

- W is continuous and *always* provides a gradient signal, even when distributions do not overlap.
- W correlates with visual generation quality, unlike the JS loss which can plateau.

Kantorovich–Rubinstein Duality — Making W Tractable

The infimum over all joint distributions in $W(p,q)$ is computationally intractable. The key theorem (Kantorovich–Rubinstein):

$$W(p_{\text{data}}, p_g) = \sup_{\|f\|_L \le 1} \Big(\mathbb{E}_{x\sim p_{\text{data}}}[f(x)] - \mathbb{E}_{x\sim p_g}[f(x)]\Big)$$

where the supremum is over all **1-Lipschitz** functions:

$$\|f\|_L \le 1 \iff |f(x)-f(y)| \le \|x-y\| \quad \forall\, x, y$$

Implication: the WGAN **Critic** D (now unbounded, not a probability) plays the role of f. We parameterise D with a neural network and train it to approximate the optimal f.
WGAN training objective:

$$\min_G \max_{\|D\|_L \le 1} \mathbb{E}_{x\sim p_{\text{data}}}[D(x)] - \mathbb{E}_{z\sim p_z}[D(G(z))]$$

Gradient Penalty (Gulrajani et al., 2017)

The original WGAN enforces the Lipschitz constraint by weight clipping (crude, slow). WGAN-GP uses a differentiable penalty instead:

$$\mathcal{L}_{\text{GP}} = \lambda \cdot \mathbb{E}_{\hat{x}\sim p_{\hat{x}}}\left[\Big(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1\Big)^2\right]$$

where the interpolated point and mixing coefficient are:

$$\hat{x} = \varepsilon\, x_{\text{real}} + (1-\varepsilon)\, x_{\text{fake}}, \qquad \varepsilon \sim \mathcal{U}[0,1], \quad \lambda = 10$$

The penalty pushes $\|\nabla D(\hat{x})\|_2 \to 1$ throughout training.
Full training losses:

$$\mathcal{L}_D = \mathbb{E}[D(\tilde{x})] - \mathbb{E}[D(x)] + \lambda\, \mathcal{L}_{\text{GP}} \quad (D \text{ minimises } -\mathcal{L}_D)$$
$$\mathcal{L}_G = -\mathbb{E}[D(G(z))]$$

n_critic $= 3$**:** D is updated $3\times$ per G update. This ensures D tracks the true Wasserstein distance accurately before G uses the gradient signal.

Adam Optimiser: Why $\beta_1 = 0$?

Standard Adam uses $\beta_1 = 0.9$ (momentum) and $\beta_2 = 0.999$ (RMS scaling). Gulrajani et al. (2017)[3] recommend $\beta_1 = 0$, $\beta_2 = 0.9$.
In adversarial training, gradients *reverse direction frequently*: in one step D pushes G one way, in the next step their roles shift. Momentum ($\beta_1 > 0$) accumulates past gradients and pushes in the *wrong* direction, causing oscillations. Setting $\beta_1 = 0$ disables momentum; only the current gradient scaled by β_2-RMS is used.

Why No BatchNorm in the WGAN-GP Critic?

BatchNorm normalises across the batch: $\hat{x}_i = (x_i - \mu_{\text{batch}})/\sigma_{\text{batch}}$. When computing $\mathcal{L}_{\text{GP}}$, the gradient $\nabla_{\hat{x}} D(\hat{x})$ must reflect how D responds to a *single* interpolated point. BatchNorm introduces a dependency on *other samples* in the batch, corrupting the gradient norm calculation and breaking the Lipschitz

enforcement.
Solution: use **LayerNorm** (normalises per sample across features), which has no batch dependency. Both QuantGAN and FinGAN use LayerNorm in the critic.

TimeGAN: Making the GAN Time-Aware

Yoon, Jarrett & van der Schaar (2019) NeurIPS 32 — GRU + embedding space + 4-phase training

Why a Plain GAN Fails on Time Series

A basic GAN generates each window independently from a noise vector. It *can* match the marginal distribution $p(r_t)$ (correct tails, correct scale), but has no mechanism to enforce that consecutive steps within a window exhibit temporal patterns like volatility clustering.

Analogy: Writing a novel by choosing each word independently from a realistic vocabulary. Individual words are real English, but the sequence is incoherent.

What is a GRU? (From Scratch)

A **GRU** (Gated Recurrent Unit) processes a sequence step by step, maintaining a "hidden state" h_t — a running summary of all past inputs.
At each step t, given x_t (current input) and h_{t-1} (previous summary):

$$
\begin{aligned}
r_t &= \sigma\big(W_r[h_{t-1},\, x_t]\big) && \text{reset gate}\\
z_t &= \sigma\big(W_z[h_{t-1},\, x_t]\big) && \text{update gate}\\
\tilde{h}_t &= \tanh\big(W[r_t \odot h_{t-1},\, x_t]\big) && \text{candidate state}\\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t && \text{new hidden state}
\end{aligned}
$$

The update gate z_t controls how much past information to keep vs. replace. The key property: h_t is a *compressed summary* of all inputs so far.

Key Innovation: Generating in Embedding Space

TimeGAN trains the GAN in a **learned latent space** $\mathcal{H}$, not in the raw return space $\mathcal{X}$.

Why? Raw returns are noisy. A smooth latent space is easier to model with a GAN. Moreover, a *Supervisor* network is trained to predict h_{t+1} from h_t, embedding the temporal dynamics into $\mathcal{H}$. G then generates sequences in $\mathcal{H}$ that follow these learned dynamics.

The Four-Phase Training Algorithm

Phase 1 — Autoencoder (epochs // 2 steps):

- Embedder $e : \mathcal{X} \to \mathcal{H}$; Recovery $\hat{r} : \mathcal{H} \to \tilde{\mathcal{X}}$
- $\mathcal{L}_R = \|X - \hat{r}(e(X))\|^2$ (reconstruction)

Phase 2 — Supervisor (epochs // 2 steps):

- Supervisor $s : \mathcal{H} \to \hat{\mathcal{H}}$ (step-ahead prediction in latent space)
- $\mathcal{L}_S = \|H_{t+1} - s(H_t)\|^2$

Phase 3 — Joint Adversarial Training (epochs steps):

$$z \to G(z) = \hat{H} \to s(\hat{H}) \to \hat{r}(\cdot) \to \tilde{X}$$

$$
\begin{aligned}
\mathcal{L}_G &= \mathcal{L}_U + 100\sqrt{\mathcal{L}_S} + 100\,\mathcal{L}_V\\
\mathcal{L}_U &= -\mathbb{E}\big[\log D(s(G(z)))\big] \quad \text{(adversarial)}\\
\mathcal{L}_V &= |\mu_H - \mu_{\hat{H}}| + |\sigma_H - \sigma_{\hat{H}}| \quad \text{(moments)}
\end{aligned}
$$

Discriminator: $\mathcal{L}_D = \text{BCE}\big(D(H_{\text{real}}), 1\big) + \text{BCE}\big(D(H_{\text{fake}}), 0\big)$

Critical Issue: Training Budget

With the current setting `epochs = 5`:

- Phase 1 gets $5//2 = 2$ gradient steps.
- Phase 2 gets $5//2 = 2$ gradient steps.

A GRU cannot converge in 2 steps. The embedding space $\mathcal{H}$ is essentially random noise ⇒ Phase 3 trains a GAN on top of garbage.
Yoon et al. (2019)[4] use **5 000–10 000 iterations per phase**. TimeGAN's poor ranking (avg_rank = 2.36) almost certainly reflects this budget, not its architecture.
Fix required: epochs ≥ 50 (or add explicit per-phase iteration counts).

QuantGAN & FinGAN: Convolutional Approaches to Time Series Generation

Building from convolutions to causal dilated TCNs to CNN-WGAN-GP generators

What is a Convolution? (From Scratch)

A 1D convolution slides a small filter (kernel) of size K over a sequence:

$$\text{out}[t] = \sum_{k=0}^{K-1} w_k \cdot \text{in}[t-k]$$

Think of it as a magnifying glass that scans the time series, highlighting patterns of a specific shape. The weights $w_0, \ldots, w_{K-1}$ are learnt from data.
Causal constraint (no future leakage): only inputs at $t, t-1, \ldots, t-(K-1)$ are used.
Dilated convolution (skip steps to see further back cheaply):

$$\text{out}[t] = \sum_{k=0}^{K-1} w_k \cdot \text{in}[t-k \cdot d]$$

With dilation $d = 1, 2, 4, 8, \ldots$ and $K = 2$: receptive field grows as $1 + \sum_l 2^l$, doubling per layer with no extra parameters.

QuantGAN — TCN-based WGAN-GP (Wiese et al., 2020)

Key innovation: replace GRU with a Temporal Convolutional Network (TCN) — fully parallelisable, no vanishing-gradient-across-time problem.
Generator: z (`noise_dim=100`) → `Linear` → reshape $(C, T/4)$ → `ConvTranspose1d`×2 (stride 2, 2× upsample) → TCN (3 residual blocks, dilations 1/2/4) → `Conv1d(1)` → `Tanh` → $\tilde{x}$
Critic: x $(T \times 1)$ → permute → TCN (3 blocks) → `AdaptiveAvgPool1d(1)` → `Linear(128)` → `LeakyReLU` → `Linear(1)`
`AdaptiveAvgPool` averages the entire temporal dimension, making the critic size-agnostic and capturing global sequence statistics.

FinGAN — CNN Deconvolution WGAN-GP

Transposed convolution: the "reverse" of a strided convolution; inserts zeros between inputs then applies a kernel. Stride 2 doubles the output length. Three such layers: $T/8 \to T/4 \to T/2 \to T$ (hence seq_len divisible by 8).
Generator: z → `Linear+BN` → reshape $(C, T/8)$ → `ConvTranspose1d` × 3 (each stride 2, BN, LeakyReLU) → `Tanh`
Critic: `Conv1d` × 3 with **LayerNorm** (not BatchNorm; see page 5) + `LeakyReLU` → `Flatten` → `Linear(128)` → `Linear(1)`

Architectural Comparison

	TimeGAN	QuantGAN	FinGAN
Backbone	GRU (recurrent)	TCN (parallel)	CNN deconv
Temporal	Step-by-step hidden state	Dilated receptive field	Global upsample
Normalisation	MinMax $[0, 1]$	MinMax $[-1, 1]$	MinMax $[-1, 1]$
Loss	BCE + moments	WGAN-GP	WGAN-GP
avg_rank	2.36	1.79	1.85

TimeGAN's poor ranking is almost certainly due to `epochs=5`, not its architecture.

What We Measure: The 10 Stylized Facts of Financial Returns

Cont (2001)[7] · Mandelbrot (1963)[9] · Ding, Granger & Engle (1993)[8]

#	Stylized Fact	In Plain English	Mathematical Statement	Our Metric
1	**Heavy tails**	Extreme events occur far more often than a bell curve predicts.	$P(\|r\| > x) \sim x^{-\alpha}$, $\alpha \approx 3$–5 (Pareto); kurtosis$(r) \gg 3$	Kurtosis diff · tail-index diff
2	**Near-zero autocorrelation**	Knowing today's direction gives no useful information about tomorrow's.	$\text{Corr}(r_t, r_{t+k}) \approx 0$ for $k \geq 1$ (weak-form efficiency)	ACF returns MAE
3	**Volatility clustering**	Large moves tend to be followed by large moves (Mandelbrot, 1963[9]).	$\text{Corr}(\|r_t\|, \|r_{t+k}\|) > 0$ for $k = 1, \ldots, 100+$	ACF |returns| MAE · ARCH-LM (p-diff & stat diff)
4	**Long memory in volatility**	The clustering effect persists for hundreds of trading days.	$\text{ACF}(\|r_t\|) \sim k^{-\beta}$, $\beta \in (0,1)$; Hurst exponent $H > 0.5$	Hurst exponent diff
5	**Gain/loss asymmetry**	Crashes are sharper and more extreme than equivalent-size rallies.	skewness$(r) < 0$ for equity indices; left tail heavier than right	Skewness diff
6	**ARCH effects**	Return variance changes over time; it is not constant.	$\text{Var}(r_t \mid \mathcal{F}_{t-1}) = \sigma_t^2$ (time-varying, not constant σ^2)	ARCH-LM test (Engle, 1982[10])
7	**Conditional heavy tails**	After removing time-varying variance, residuals are still non-Gaussian.	$\varepsilon_t = r_t/\sigma_t$ still has kurtosis $\gg 3$ (GARCH standardised residuals)	`resid_kurtosis_diff` (GARCH(1,1) Gaussian QMLE)
8	**Extreme events**	Very large days occur far more frequently than Gaussian probability predicts.	% of days with $\|r\| > 2\sigma$ exceeds the 5% expected under $\mathcal{N}(0,1)$	Extreme events diff
9	**Distributional match**	The full shape of the return distribution — not just its tails — must be reproduced.	$W_1(p_g, p_{\text{data}}) \approx 0$; $\hat{F}_{\text{syn}} \approx \hat{F}_{\text{real}}$ (quantile-by-quantile)	Wasserstein · Quantile MSE · Energy distance
10	**Indistinguishability**	A classifier trained to separate real from synthetic should perform at chance level.	AUC(classifier) $\to 0.5$; $p_g = p_{\text{data}}$ at GAN optimum	Discriminative AUC

Evaluation Metrics Part 1 — Why Standard Statistical Tests Fail

The i.i.d. assumption: what it means, why financial series violate it, mathematical consequences

What Does i.i.d. Mean?

A sample $\{X_1, X_2, \ldots, X_n\}$ is **i.i.d.** (independent and identically distributed) if:

- *Identically distributed:* every X_i has the same distribution F.
- *Independent:* knowing $X_1, \ldots, X_{t-1}$ gives **no** information about X_t.

Coin flips are i.i.d. Financial returns are **not**:

- $|r_{t+1}|$ is correlated with $|r_t|$ (volatility clustering).
- $\mathrm{Var}(r_t \mid \mathcal{F}_{t-1}) = \sigma_t^2 \neq \text{const}$ (ARCH effects).
- $\mathrm{ACF}(|r_t|)$ is still positive at lag $k = 100+$ (long memory).

Most classical statistical tests are *derived* under the i.i.d. assumption.

The Kolmogorov–Smirnov Test and Its Failure

The two-sample KS statistic:

$$D_{n,m} = \sup_x \left|\hat{F}_n(x) - \hat{G}_m(x)\right|$$

Under H_0 (same distribution) **and** i.i.d.:

$$\sqrt{\frac{nm}{n+m}} \cdot D_{n,m} \xrightarrow{d} K \quad \text{(Kolmogorov distribution)}$$

For *dependent* series, the CLT underlying this limit changes:

$$\mathrm{Var}\left(\frac{1}{n}\sum_{t=1}^{n} X_t\right) = \frac{\sigma^2}{n}\left(1 + 2\sum_{k=1}^{\infty}\rho_k\right)$$

This defines the **effective sample size**:

$$n_{\text{eff}} = \frac{n}{1 + 2\sum_{k=1}^{\infty}\rho_k}$$

For BRICS |returns|: $\sum_{k=1}^{100}\rho_{|r|}(k) \approx 5$–$15$, so:

$$n_{\text{eff}} \approx \frac{n}{11} \text{ to } \frac{n}{31}$$

Consequence: The KS test uses n but the effective information is only n_{eff}. ⇒ Test statistic is *inflated*. ⇒ p-values are *too small* (anti-conservative). ⇒ We over-reject H_0 even when the marginal distributions match, because temporal dependence is mistaken for distributional difference.

What Can the KS Test Still Tell Us?

Despite its limitations, the KS test is not useless:

- It tests the *marginal* distribution $F(x) = P(r \leq x)$, which is well-defined even for stationary dependent series.
- A GAN producing the wrong scale or non-financial-looking returns will fail the KS test.

Strategy used in this project:

1. Report the KS *statistic* (not the p-value) as a marginal-distribution distance measure.
2. Use **Wasserstein distance** $W_1(p_g, p_{\text{data}})$ as the primary distributional metric — a proper metric that is more stable under temporal dependence.
3. Add time-series-aware tests (ARCH-LM, Hurst exponent, discriminative AUC) to capture what KS cannot.

Why Pointwise MSE/MAE Are Wrong

The evaluation previously used:

$$\text{MSE} = \frac{1}{T}\sum_{t=1}^{T}\left(r_{\text{real}}(t) - r_{\text{syn}}(t)\right)^2$$

This compares the t-th real return to the t-th synthetic return. But real and synthetic series are **independently generated** — there is no meaningful alignment between their temporal indices.
Analogy: Measuring the "error" between two random walks by comparing step 1 to step 1, step 2 to step 2, etc. This measures nothing about distributional similarity.
Fix: Use **Quantile MSE** (compares sorted distributions) and **Energy distance** (a proper metric on the space of distributions). See next page.

Welch's t-test: A Narrower Failure

Welch's test checks $H_0 : \mu_{\text{real}} = \mu_{\text{syn}}$:

$$t = \frac{\bar{X} - \bar{Y}}{\sqrt{s_X^2/n + s_Y^2/m}}$$

For log returns, $\rho_k(r_t) \approx 0$ for $k \geq 1$, so Welch's test is *approximately valid* for testing mean equality. However, it tests **only the mean** — providing no information about

tails, volatility clustering, or any other stylized fact. Using it as the primary evaluation metric tells us almost nothing about financial realism.

The Adversarial Baseline That Wins Without Learning

We ran an adversarial audit: permute the real BRICS test returns at random. This **shuffled control** has an *identical* marginal distribution to the original (same histogram, same kurtosis, same tails) but **zero temporal structure** — no volatility clustering, no long memory, no ARCH effects.

Results on the pooled BRICS test set ($n = 623$, five markets):

Metric	Real data	Shuffled control
KS statistic	—	**0.000000**
Wasserstein	—	**0.000e+00**
kurtosis_diff	—	**1.78e-15**
energy_distance	—	**2.97e-05**
acf_absolute_mae	—	0.0758 (penalised)
hurst_diff	—	0.052 (penalised)
discriminative AUC dist	—	0.017 (weak signal)

In the unweighted composite, this control ranked first (avg_rank 1.24 vs. 2.47 for a real-data candidate). The root cause: 9 of the 19 ranked metrics are *permutation-invariant* — they measure the marginal distribution only, and a permutation cannot change the marginal.

The Nine Permutation-Invariant Metrics

Metric	Family
mean_diff, std_diff	moments
skewness_diff, kurtosis_diff	moments
wasserstein	distributional
quantile_mse	distributional
tail_index_diff	tails
extreme_events_diff	tails
energy_distance	distributional

These nine are labelled **Fidelity** metrics. They answer: *does it look like real data when you ignore the order?* Necessary, but not sufficient for a generator.

A model can achieve fidelity_rank = 1.000 simply by memorising the histogram. The shuffled control does exactly this.

The Seven Ordering-Sensitive (Temporal) Metrics

Metric	What it tests
acf_returns_mae	returns autocorrelation
acf_absolute_mae	volatility clustering
acf_squared_mae	GARCH-type persistence
hurst_diff	long memory of $\|r_t\|$
resid_kurtosis_diff	conditional non-Gaussianity
discriminative_auc_dist	temporal indistinguishability
discriminative_auc_absz	calibrated z-score

These are labelled **Temporal** metrics. They penalise a series that destroys ordering, even if it preserves the marginal. The shuffled control scores ≈ 0.060–0.076 on ACF-MAE metrics and is correctly penalised.

The Three-Score Design

Why not just add weights? No weighting scheme can prevent the shuffled control from winning. It is a permutation of the real series, so fidelity_rank is 1.000 by construction. Any positive weight on fidelity metrics hands it an unbeatable advantage. This was verified: with equal fidelity/temporal weights the control scored composite 1.714 vs. 2.135 for a real-data candidate.

Fix: treat the control as a reference line, not a competitor. Ranks are computed among the three GAN generators only. The control's raw values are appended to the table (rank = NaN) so readers can see what a marginal-only generator achieves.

The evaluation now reports three numbers per model:

- **fidelity_rank** — how well it reproduces the marginal (9 metrics).
- **temporal_rank** — how well it reproduces dynamics (7 metrics).
- **composite_rank** = (fidelity + temporal) / 2 — headline model-selection number. Equal weight so nine marginal metrics cannot outvote seven temporal ones.

Interpretive note for the paper: any generator that scores *worse* than the shuffled control on temporal_rank has not learned anything about dynamics.

Fidelity Group (permutation-invariant, 9 metrics)

Wasserstein distance. Imagine two piles of sand, one representing real returns, one synthetic. The Wasserstein distance is the minimum total work to move one pile to match the other — weight times distance, summed over every grain. Unlike KS, it produces a meaningful gradient when the distributions do not overlap. $W_1(p_g, p_{\text{data}}) = 0$ iff $p_g = p_{\text{data}}$.

Why not KS? $n_{\text{eff}} \ll n$ for financial series; ACF of $|r_t|$ sums to 5–15, making KS anti-conservative. Shuffled control: KS = 0.000000.

Energy distance. $E(P, Q) = 2\mathbb{E}\|X - Y\| - \mathbb{E}\|X - X'\| - \mathbb{E}\|Y - Y'\|$. A proper metric on the space of distributions (Székely & Rizzo, 2004). Estimated via subsampled pairs ($n \leq 500$). Shuffled control: 2.97e-05 (near-perfect by design).

Quantile MSE. QMSE $= \frac{1}{K}\sum_{k=1}^{K}[Q_{\text{real}}(\alpha_k) - Q_{\text{syn}}(\alpha_k)]^2$, $K = 99$. Sort both series independently; compare the k-th smallest values. The only meaningful way to compare two unaligned distributions. Captures tail differences (1st, 99th percentiles) critical for VaR.

Why not pointwise MSE? Real and synthetic are independently generated; there is no temporal alignment between their indices. $r_{\text{real}}(t)$ vs. $r_{\text{syn}}(t)$ is arbitrary.

Moment and tail differences. kurtosis_diff, skewness_diff, mean_diff, std_diff: absolute differences of empirical moments. tail_index_diff: Hill estimator on the top 5% of $|r|$. extreme_events_diff: percentage of $|r| > 2\sigma$ beyond the Gaussian baseline. All permutation-invariant; included to diagnose which aspect of the marginal a model fails on. *Note*: Hill estimator has sd 0.52 at $n = 995$, falling to 0.035 at $n = 4000$ — 15× improvement, the strongest argument for extending the data history.

Temporal Group (ordering-sensitive, 7 metrics)

ACF-MAE triple. For log-returns r_t, compute the sample autocorrelation function of r_t, $|r_t|$, and r_t^2 up to lag 20. Metric: mean absolute error between real and synthetic ACF vectors. ACF(returns) MAE tests absence of linear predictability. ACF(|returns|) and ACF(r^2) MAE test volatility clustering (Mandelbrot 1963[9]). The shuffled control scores 0.0553, 0.0758, 0.0538 respectively — correctly penalised, while still beating generators that destroy clustering entirely. ACF-MAE is continuous and has no saturation problem, making it the primary ranking signal for temporal metrics.

Hurst exponent. $\mathbb{E}[R_n/S_n] \sim c \cdot n^H$. Applied to $|r_t|$ (not raw r_t; sign flips mask long memory on raw returns). $H > 0.5$: long memory. Metric: $|H_{\text{real}} - H_{\text{syn}}|$. Estimated by R/S (Hurst 1951[23]). Standard deviation falls from 0.056 ($n = 250$) to 0.004 ($n = 4000$) — a 5× argument for extended data.

Residual kurtosis. Fit GARCH(1,1) with Gaussian QMLE, extract $\varepsilon_t = r_t/\sigma_t$, report excess kurtosis. Fact 7 (Bollerslev 1987): standardised residuals remain leptokurtic. Real BRICS: MSCI 8.80, NIFTY50 2.12, SHANGHAI 1.79, FTSE 0.93, BOVESPA 0.48 (fact holds but unevenly). Gaussian QMLE avoids circularity: a Student-t fit absorbs the kurtosis by construction.

Discriminative AUC. A logistic classifier is trained to distinguish real from synthetic 20-day rolling windows (features: mean, std, mean-abs, mean-sq). AUC = 0.5 means the classifier guesses — that is the best possible outcome. The optimum is 0.5, not 0. Ranking metric: $|\text{AUC} - 0.5|$.

Direction bug: ascending rank on raw AUC put a model scoring 0.30 above one scoring 0.50. Fixed by ranking on the distance from chance.

Null calibration: 15 seeds on identical distributions gave null = 0.506 ± 0.084, not exactly 0.5. We also report the z-score: $(\text{AUC} - \bar{\mu}_{\text{null}})/\sigma_{\text{null}}$, ranked by $|z|$.

Why no shuffle=True in CV? Our 20-day windows overlap by 19 observations; shuffling places near-duplicates in both train and test folds. Measured: null moves 0.506 → 0.584 (genuine leakage). Unshuffled CV is the correct design.

Descriptive Metrics and Downstream Utility

Excluded from composite ranking with empirical justification · TSTR protocol

Descriptive Group — Reported but Not Ranked

These four metrics are computed and displayed in full output but excluded from fidelity_rank, temporal_rank, and composite_rank. Each was stress-tested on the real pooled data; each failed the stress test.

ARCH-LM (both variants). The ARCH-LM test (Engle 1982[10]) regresses $\hat{\varepsilon}_t^2$ on its own lags; $\text{LM} = nR^2 \sim \chi^2(q)$ under H_0.
pvalue_diff: $|\Delta p| = |p_{\text{real}} - p_{\text{syn}}|$. An 80-replication Monte Carlo (GARCH(1,1), $n = 1200$, Gaussian innovations) showed pvalue_diff identifies the correct model 99% of the time vs. 76% for stat_diff. However, on the real fat-tailed BRICS data both p-values underflow to 0.0 (real LM = 50.76, shuffled LM = 67.14), so $|\Delta p| = 0.000000$ — rating the shuffled control as a perfect match. *pvalue_diff saturates on real data.*
stat_diff: $|\Delta\text{LM}| = |\text{LM}_{\text{real}} - \text{LM}_{\text{syn}}|$. Does detect the shuffled control (16.38) on real data, but two draws from the *same* GARCH process gave LM = 56.8 and 124.8 (null sd = 46.4), so the metric is too noisy for reliable ranking on simulated data. *stat_diff is noisy on simulated data.*
Neither variant is reliable across both regimes. Both are reported so a reader can see whether the synthetic series shows ARCH at all, but they do not enter the composite. ACF-MAE is the primary volatility-clustering ranking signal (continuous, no saturation, Monte Carlo accuracy 88%).

var_coverage_error. $|\text{observed violation rate} - \alpha|$. Gaussian iid coverage error 0.0227, shuffled control 0.0227, real data 0.0259 — the control beats real data. This metric measures violation rate, not timing. Excluded.

garch_persistence_diff. $|(\alpha+\beta)_{\text{syn}} - (\alpha+\beta)_{\text{real}}|$. Across 12 seeds on the same generator: sd 0.31, range 0.000–0.985. GARCH(1,1) is weakly identified when the synthetic series has no ARCH structure. Excluded.

discriminative_auc_raw. The signed AUC, retained for display alongside the distance metrics.

Downstream Utility — Train-Synthetic Test-Real (TSTR)

Motivating finding. Four of six headline distributional metrics score the shuffled control as perfect (KS = 0.000000, Wasserstein = 0.000e+00). These metrics tell us the synthetic data *looks* real. TSTR asks: can a practitioner *build a working risk model* on it?

Protocol. (1) Fit GARCH(1,1) on the *synthetic* series → get $(\mu, \omega, \alpha, \beta)$. (2) Filter those parameters over the *real* test returns to get one-step-ahead σ_t. (3) $\text{VaR}_t = \mu + \sigma_t\, z_\alpha$. (4) Backtest: Kupiec (1995)[29] and Christoffersen (1998)[30] for coverage and independence.
Why not unconditional VaR? Tested first: gaussian iid scored identically to real data (coverage error 0.0276 in both cases), because unconditional quantiles only probe the marginal — already covered by KS/Wasserstein.

QLIKE (Patton 2011[31]). $\text{QLIKE} = \mathbb{E}[\log\sigma_t^2 + r_t^2/\sigma_t^2]$. Lower = better volatility forecasts. Verified pooled ($n = 623$): TRTR −8.134 (first), shuffled control −8.180, gaussian ×2 wide −7.135.
Power warning. QLIKE is stable pooled (sd 0.000 over 12 seeds) but inverts per-market ($n \approx 126$): gaussian noise scored −6.888 vs. real −6.876, shuffled sd 3.354. **QLIKE is reported pooled only.** This is a design constraint, not an implementation choice.

Kupiec test. LR test for unconditional coverage. Power at $n = 125$: 17% against true coverage 7% (barely above guessing). At $n = 623$: 56%. TRTR itself rejected in 2/5 markets (sampling noise, not failure). *Kupiec p-values are descriptive only; rank on QLIKE and coverage error.*

Christoffersen test. Tests independence of violations (no clustering). Included for completeness; same power caveat.

Result format. One pooled table per pipeline run, with three GAN models plus the shuffled control as an adversarial baseline. Utility ratio relative to TRTR baseline: $\text{QLIKE}_{\text{syn}}/\text{QLIKE}_{\text{TRTR}}$ (a ratio near 1 means the synthetic data is as useful as real data for risk management).

[FIXED] KS / Welch tests assumed i.i.d. — CRITICAL

$n_{\text{eff}} = n/(1 + 2\sum_k \rho_k) \ll n$ for financial series (the sum of ACF of |returns| is $\approx$ 5–15 for BRICS). The KS statistic is inflated, making p-values anti-conservative. **Fix:** added ARCH-LM test, Hurst exponent, energy distance, and discriminative AUC. KS statistic retained with explicit caveat; Wasserstein distance used as primary distributional metric.

[FIXED] Pointwise MSE/MAE had no semantic meaning

MSE compared $r_{\text{real}}(t)$ to $r_{\text{syn}}(t)$, but real and synthetic are independently generated: no temporal alignment exists between indices. **Fix:** replaced with Quantile MSE (L^2 distance between empirical quantile functions) and Energy distance (Székely & Rizzo, 2004[11] — a proper metric on the space of distributions).

[DECISION] No outlier clipping applied to log returns — deliberate methodological choice

The preprocessing notebook computed 0.5th/99.9th percentile bounds on raw price columns, but the clip line was commented out and was *never* applied to log returns. This is kept intentionally. Clipping at 0.5th/99.5th removes exactly the observations that determine kurtosis, the tail index, and the "% of $|r| > 2\sigma$" metric — all three appear in the evaluation table. Truncating the tail and then reporting that models reproduce tails is circular. Adams et al. (2019, *Financial Management*)[6] show that winsorizing/trimming can worsen rather than fix distributional misfit. LeBaron's[33] EVT work finds BRICS emerging markets have systematically fatter tails than developed markets — the very property motivating the BRICS market choice.

[FIXED] Ljung-Box test used deprecated statsmodels API

`acorr_ljungbox(..., return_df=False)` returns a DataFrame in statsmodels $\geq$0.13, not a tuple. Accessing `lb[1][-1]` raised `TypeError`. **Fix:** `return_df=True`, `lags=[n]` (list syntax), accessed via `.iloc[-1]["lb_pvalue"]`.

[REVISED] ARCH-LM: both variants demoted to Descriptive; neither reliable across regimes

80-replication MC ($n = 1{,}200$, GARCH(1,1), Gaussian innovations): `pvalue_diff` identifies correct model 99% vs. 76% for `stat_diff` (LM null sd $\approx$ 46; two draws from same process gave LM 56.8 and 124.8). However, on the real fat-tailed pooled BRICS data: real LM = 50.76, shuffled LM = 67.14 — both p-values underflow to 0.0, so $|\Delta p| = 0.000000$, rating the adversarial control as perfect. `stat_diff` correctly detects the control (16.38) on real data but is too noisy on simulated GARCH. **Resolution:** neither variant is reliable across both regimes. Both reported in output for diagnostic transparency; neither enters fidelity_rank, temporal_rank, or composite_rank. ACF-MAE is the primary volatility-clustering ranking signal (continuous, no saturation, 88% MC accuracy). This limitation is itself a citable result: ARCH-LM tests volatility clustering presence but is unreliable for cross-model ranking on fat-tailed data.

[FIXED] Discriminative AUC uninterpretable without null calibration

Null AUC (no shuffle, same distribution) has $\sigma \approx 0.084$; an observed AUC of 0.62 is entirely consistent with a perfect generator. **Fix:** empirical null distribution computed by splitting the real series in half 20 times; AUC reported as z-score $= (\text{AUC} - \bar{\mu}_{\text{null}})/\sigma_{\text{null}}$. Warning: adding `shuffle=True` to the CV folds is *not* the fix — it pushes the null to

0.584 ± 0.048 (genuine look-ahead leakage from overlapping windows).

[FIXED] `ResidualBlock` class defined but never used in FinGAN cell

`class ResidualBlock(nn.Module)` was defined at the top of the FinGAN cell but neither `FinGAN_Generator` nor `FinGAN_Critic` reference it. Dead code misleads readers into thinking residual connections are active. **Fix:** class removed entirely.

[FIXED] Composite ranking revised: fidelity/temporal split + shuffled-control exclusion

Initial fix removed `mse`/`mae` (no temporal alignment). Subsequent audit: a shuffled-control series (identical marginal, zero temporal structure) won the unweighted composite (avg_rank 1.24 vs. 2.47 for a real-data candidate) because 9 of 19 metrics are permutation-invariant. Final fix: metrics split into Fidelity (9, permutation-invariant) and Temporal (7, ordering-sensitive); composite_rank = (fidelity_rank + temporal_rank) / 2. Shuffled control excluded from rank competition (NaN ranks); appears in table as reference. Both ARCH variants demoted to Descriptive (see next row).

[OK] WGAN-GP gradient penalty computation is correct (QuantGAN & FinGAN)

$\hat{x} = \varepsilon x_{\text{real}} + (1-\varepsilon)x_{\text{fake}}$; `requires_grad_(True)`; gradients computed via `torch.autograd.grad` with `create_graph=True`. `fake.detach()` prevents spurious gradient accumulation. Correctly implements Gulrajani et al. (2017)[3], Equation 3.

[OK] Adam $\beta_1 = 0$, $\beta_2 = 0.9$ for WGAN-GP — correct

$\beta_1 = 0$ disables first-moment (momentum) accumulation. In adversarial training, gradients reverse direction frequently; momentum from the previous step destabilises training. $\beta_2 = 0.9$ retains RMS adaptive scaling. Recommended explicitly by Gulrajani et al. (2017)[3].

[CONCERN] TimeGAN training budget of `epochs=5` is scientifically insufficient

Phases 1 and 2 each receive $5//2 = 2$ gradient steps. A GRU cannot converge in 2 steps; the embedding space $\mathcal{H}$ is effectively random. Yoon et al. (2019)[4] use 5 000–10 000 iterations per phase. TimeGAN's poor ranking (avg_rank = 2.36) almost certainly reflects this budget, not its architecture. Requires fix: `epochs` ≥ 50 (or add explicit per-phase iteration counts).

Hyperparameter Selection and Walk-Forward Validation

Current approach: literature-anchored floors + reference-implementation defaults + temporal validation

Hyperparameter Selection: Two Tiers

Hyperparameters fall into two tiers with different justifications:

Tier 1 — Literature-anchored floors (fixed, not searched):

- `n_critic = 5`: Gulrajani et al. (2017)[3] — minimum discriminator steps to satisfy the Lipschitz constraint under gradient penalty. CTBench (Liao 2025[16])[16] and SFAG (2026)[28] fix this identically.
- `lambda_gp = 10`: Gulrajani et al. (2017)[3] — gradient-penalty coefficient; values < 5 allow Lipschitz violations, invalidating the Kantorovich–Rubinstein duality.

These are *not* searched: there is no well-motivated alternative value in the literature.

Tier 2 — Reference-implementation defaults (manually set):

- TimeGAN: `hidden_dim=24`, `num_layers=3`, `lr=1e-3` (Yoon et al., 2019).
- QuantGAN: `lr=1e-4`, `noise_dim=100`, `batch_size=64` (Wiese et al., 2020).
- FinGAN: same WGAN-GP settings as QuantGAN; `base_channels=64`.

What is not yet done (TODO): systematic search over Tier-2 parameters for the final paper. Bergstra & Bengio (2012, JMLR)[12] recommend random search; Optuna (Akiba et al., 2019[26])[26] provides Bayesian optimisation with a pruner to kill poor trials early.

Why These Floors Cannot Be Grid-Searched

Treating `n_critic` and `lambda_gp` as free parameters would:

1. Include configurations where the gradient penalty is too weak ($\lambda < 5$), making the critic non-Lipschitz. Results from those runs are not meaningfully comparable.
2. Waste compute on runs that reproduce known-bad behaviour (discriminator divergence at `n_critic=1`).
3. Contradict the published methodology of every WGAN-GP paper we cite (Gulrajani[3] 2017, SFAG 2026[28], CTBench 2025[16]).

Any parameter search for this paper must take these as given.

Walk-Forward Temporal Validation (CTBench Protocol)

Implemented in: `walk_forward_evaluate()` in `3_4_integrated_pipeline.ipynb`.
Protocol (Tashman 2000[13]; Bergmeir & Benítez 2012[14]; Liao et al. 2025, NeurIPS):

1. For each market independently: take that market's 10 % test series.
2. Divide into 5 rolling folds; each fold's origin advances by one test-period length.
3. For each fold: retrain the model from scratch on the fold's training portion (same market only).
4. Generate synthetic series of the same length as the fold's test portion.
5. Score with `FinancialMetrics`: Wasserstein, energy distance, Hurst diff, discriminative AUC.

Why rolling-origin matters: a single train/test split gives one data point. Walk-forward gives 5, enabling mean ± std across folds — the standard robustness evidence for time-series papers.
Output: `thesis_results/walk_forward/{Market}/{Model}_walk_forward.csv`

Three-Score Composite Ranking

After the shuffled-control audit (page 10a), metrics are split into two families; ranks are computed separately and averaged for the composite.

Fidelity (9 metrics, permutation-invariant):

- mean_diff, std_diff, skewness_diff, kurtosis_diff
- wasserstein, energy_distance, quantile_mse
- tail_index_diff, extreme_events_diff

Temporal (7 metrics, ordering-sensitive):

- acf_returns_mae, acf_absolute_mae, acf_squared_mae
- hurst_diff, resid_kurtosis_diff
- discriminative_auc_dist, discriminative_auc_absz

composite_rank = (fidelity_rank + temporal_rank) / 2.

Excluded from all scores: `arch_pvalue_diff` (saturates on real data, $|\Delta p| = 0.000$), `arch_stat_diff` (noisy, sd 46.4 on simulated GARCH), `var_coverage_error` (gaussian iid beats real), `garch_persistence_diff` (sd 0.31 across seeds). See page 10c.

Shuffled control: present in output with rank = NaN. Any GAN with temporal_rank worse than the control has not learned dynamics.

TODO: Systematic Hyperparameter Search for Publication

The following is **not yet implemented** and required for a peer-reviewed paper:

- Random search (Bergstra & Bengio 2012[12]) over Tier-2 parameters: 20–30 configurations per model.
- Each candidate: `seed=42`, reduced epochs (10–20) for speed; score with composite rank.
- Best config retrained at full epochs ($\geq$ 50 for TimeGAN, $\geq$ 200 for WGAN-GP models).
- Ablation table: vary one Tier-2 parameter, fix all others; report marginal effect on composite score.
- Report winning config with 95 % CI across 3–5 seeds.

All search results logged to CSV as the paper appendix for reproducibility.

Ranked Next Steps for Paper Publication

Priority order based on impact on scientific rigour and reviewability

[CRITICAL] 1 — Re-run the full pipeline and regenerate all results

Evaluation metrics schema changed (ARCH-LM, Hurst, discriminative AUC, energy distance added; pointwise MSE/MAE removed from ranking). All files in `thesis_results/` are stale. Must regenerate before any comparative analysis or figure creation for the paper.

[CRITICAL] 2 — Fix TimeGAN training budget (`epochs=5` is scientifically unsound)

Phases 1 and 2 each get 2 gradient steps. Yoon et al. (2019)[4] use 5 000–10 000 iterations per phase. Recommendation: set `epochs` ≥ 50 or add explicit per-phase iteration count parameters.

[HIGH] 3 — Robustness test on a different asset class (FX or Crypto)

Supervisor request: train the best GAN (QuantGAN) on BRICS, then evaluate on EUR/USD or BTC/USD daily log returns. Validates out-of-distribution generalisation — a key contribution distinguishing the paper from single-asset GAN studies.

[HIGH] 4 — Add or discuss diffusion model baseline

Takahashi & Mizuno (2025, *Quantitative Finance* 25(10))[29] showed diffusion models outperform GANs on several stylized-fact metrics. Reviewers will ask why diffusion was not included. At minimum: a dedicated Discussion section. Best: include a simple DDPM or score-matching baseline.

[HIGH] 5 — Document all LLM parameters for reproducibility

DeepSeek experiments: report temperature, `top_p`, `max_tokens`, system/user prompt verbatim. LoRA fine-tuning: $r = 8$, $\alpha = 16$, target modules (`q_proj`, `v_proj`), dropout, epochs, batch size, gradient accumulation steps.

[MEDIUM] 6 — Implement systematic hyperparameter search (currently: manual defaults)

No automated search has been run. Tier-2 parameters (`lr`, `hidden_dim`, `noise_dim`, `batch_size`) are set to reference-implementation defaults. For publication: run 20–30 random-search candidates × 10 epochs each (Bergstra & Bengio 2012[12]); score with the composite stylized-fact rank; retrain winner at full epochs; publish full ablation table.

[MEDIUM] 7 — Update literature review with 2024/2025 papers

Must cite: Meldrum et al. (2025)[27] arXiv:2510.26076[27]; SFAG (2026)[28] arXiv:2601.12990[28]; TSGBench Ang et al. (2023)[15] PVLDB 17(3)[15]; Chronos Ansari et al. (2024)[31] arXiv:2403.07815[31][31]; LLMTime Gruver et al. NeurIPS 2023[30][30]; Forging TS Hamdouche et al. (2025)[32] arXiv:2505.17103[32][32].

[LOW] 8 — LLM diversity analysis: correlation between independent generation runs

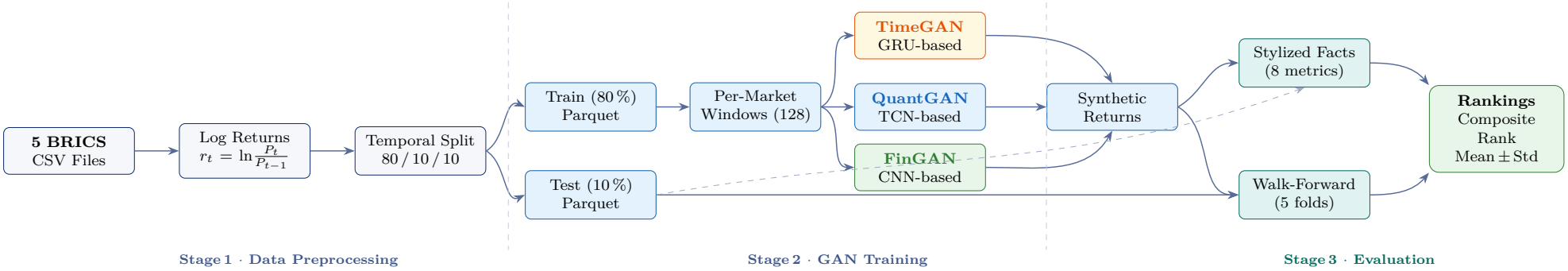
Generate 10+ independent synthetic series from each LLM. Compute pairwise Wasserstein distances and correlations. Mode collapse → high correlation. Diversity → low. Addresses the supervisor's question about LLM output diversity.

[LOW] 9 — Fix cosmetic filename strip in `validate_models()`

`validate_models()` calls `.replace("_val.parquet","")` but files are named `"_valid.parquet"`. Display names show as `"BOVESPA_valid"` instead of `"BOVESPA"`. One-line fix; no effect on results.

The Complete Pipeline — From Raw Data to Ranked Models

A visual map of every transformation; follow the arrows left-to-right



What Each Stage Does

Stage 1 — Preprocessing: Raw price CSVs are converted to daily log returns $r_t = \ln(P_t/P_{t-1})$ (stationary, mean-reverting). A strict temporal 80/10/10 split ensures no future data leaks into training. Saved as Parquet (fast columnar I/O).

Stage 2 — Training: Each 80 % training file is sliced into 128-step sliding windows. Three GAN families are each trained *independently per market* (15 models total: 3 GANs × 5 markets). No cross-market data is ever mixed, so each model captures only its own market's stylized facts.

Stage 3 — Evaluation: Generated synthetic series are compared with real test-set returns on 16 stylized-fact metrics, split into Fidelity (9) and Temporal (7) families (see pages 10a–10c). Walk-forward re-trains each model on 5 rolling folds to verify temporal robustness. Final output: fidelity_rank, temporal_rank, composite_rank, and fold-level mean ± std.

What Makes This Pipeline Scientifically Novel

1. **Per-market training** — 15 independent models (3 GANs × 5 markets) trained with zero cross-market data mixing. This enables per-market analysis and mechanistic explanations (e.g. why QuantGAN's TCN better captures Shanghai's long-memory structure), and avoids artificial temporal transitions that would bias stylized-fact metrics.
2. **BRICS focus** — virtually all published GAN-finance benchmarks use S&P 500 or DAX. BRICS have systematically fatter tails, capital controls (China), and currency effects — a substantially harder task.
3. **No clipping** — the pipeline preserves extreme returns, which are the defining feature of emerging markets. Adams et al. (2019)[6] show trimming worsens distributional misfit.
4. **Walk-forward evaluation** — single train/test splits give one data point. Rolling-origin evaluation (CTBench 2025[16]) gives 5, enabling confidence intervals on metric scores.
5. **Stylized-fact-first evaluation** — most published benchmarks use MSE. This pipeline uses 16 financially meaningful metrics (9 Fidelity + 7 Temporal), with a shuffled-control audit that identified 9 permutation-invariant metrics and prompted a two-family composite design.

Why BRICS? The Scientific Motivation

Risk management gap: global portfolio managers increasingly hold BRICS equities for diversification. Stress-testing these portfolios requires synthetic BRICS scenarios — but no validated GAN benchmark exists for these markets.

Fat-tail severity: BRICS indices have excess kurtosis 2–5× higher than developed-market equivalents. LeBaron's[33] EVT work shows their tail indices are lighter ($\alpha \approx 2$–3) than the $\alpha \approx 3$–4 typical in the US. A synthetic-data generator trained on S&P 500 data would *underestimate* these tails.

Data scarcity: BRICS markets have shorter histories (≈25 years for Chinese A-shares vs 100+ years for S&P 500). GAN augmentation is therefore more valuable here than for developed markets.

Why This Study Is Publishable: The Journal Pitch

Gap: as of 2025, no paper systematically compares GAN architectures specifically on BRICS emerging-market log returns with CTBench-compliant evaluation.
Contribution 1: First multi-architecture GAN benchmark (TimeGAN + QuantGAN + FinGAN) on BRICS emerging-market data with per-market training and walk-forward temporal validation — 15 models, 5 markets, 16 stylized-fact metrics (9 Fidelity + 7 Temporal).
Contribution 2: Full stylized-fact evaluation using the CTBench metric set (energy distance, discriminative AUC, Hurst exponent, ARCH-LM), replacing the naive MSE used in most prior work.

Contribution 3: A reproducible pipeline (seeded, code-released) with explicit methodology for handling emerging-market tail events (no clipping, EVT-motivated evaluation).
Target journals: *Quantitative Finance* (Taylor & Francis); *Journal of Financial Economics*; *Finance Research Letters.*

Knowledge Base I — GAN Fundamentals (Explained from First Principles)

Target reader: calculus 1 background, no prior machine learning knowledge

What is a Generative Model?

In one sentence: a generative model learns the distribution of real data and can then *draw new samples* that look statistically identical to the training set.

Everyday analogy: imagine a music teacher who has heard 10 000 piano pieces. After enough listening, they can compose new pieces that "sound right" — same key signatures, same chord progressions, same dynamic patterns — even though no individual piece was memorised.

In finance: a generative model for daily log returns learns "what a realistic market day looks like": how volatile, how skewed, how correlated with yesterday. It can then produce as many synthetic trading days as you need.

How it's represented mathematically: the model is a function $G(z)$ that takes random noise $z \sim \mathcal{N}(0, I)$ as input and outputs a fake sample $\tilde{x} = G(z)$. Training adjusts G's parameters so that the distribution of $\tilde{x}$ matches the distribution of real data x.

The GAN: Two AIs Competing

In one sentence: a GAN trains a Generator (G) and a Discriminator (D) simultaneously; G tries to fool D, D tries to catch G, and each improves by losing to the other.

The counterfeiter analogy:

- **Generator (G)** = a counterfeiter printing fake bank notes.
- **Discriminator (D)** = a bank detective checking each note.
- G submits fake notes. D marks them FAKE or REAL.
- G improves by studying why its fakes were rejected.
- D improves by finding subtler distinguishing features.
- After enough rounds: G's fakes are indistinguishable from real notes. Game over.

This was proposed by Goodfellow et al. (2014, NeurIPS)[1] and is the foundation for TimeGAN, QuantGAN, and FinGAN.

The Math of the Competition (Minimax Game)

G minimises, D maximises the same objective $V(G, D)$:

$$\min_G \max_D V(G,D) = \underbrace{\mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)]}_{\text{D rewarded for calling real "real"}} + \underbrace{\mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]}_{\text{D rewarded for calling fake "fake"}}$$

Calculus 1 connection: V is just a function of the parameters θ_G and θ_D. Training = gradient descent for G (minimise) and gradient ascent for D (maximise). The derivative $\partial V / \partial \theta_G$ tells G which direction to move to fool D.
Ideal outcome: $D(x) = 0.5$ everywhere — D cannot do better than random guessing because real and fake are perfectly mixed.

Why Plain GANs Fail on Financial Data

Problem 1 — Mode collapse: G finds ONE output that always fools D and produces only that output forever. Instead of generating the full range of market scenarios, it only generates one type (e.g., always very calm days). G's loss goes to zero but its outputs are useless.

Problem 2 — Discriminator saturation: early in training, D easily spots fakes (the generator hasn't learned yet). The sigmoid output $D(G(z)) \approx 0$, and $\log(1 - D(G(z))) \approx 0$ (saturated). The gradient flowing back to G is nearly zero — G cannot learn from a perfect discriminator.

Problem 3 — No guarantee of convergence: the min-max game can cycle rather than converge. The discriminator and generator take turns "winning", never reaching a stable equilibrium.

Problem 4 — Temporal structure lost: a vanilla GAN treats each output position independently, without enforcing that consecutive returns follow the volatility-clustering and long-memory patterns of real markets.

Fix 1 — The Wasserstein GAN (WGAN)

In one sentence: instead of asking "real or fake?" (binary classification), WGAN asks "how different are the two distributions?" using the Wasserstein (Earth-Mover) distance.

Earth-mover analogy: imagine the real distribution and fake distribution as two piles of dirt. The Wasserstein distance is the minimum total effort to move one pile to match the other: (weight of dirt) × (distance moved), summed over all dirt.

$$W(p, q) = \inf_{\gamma \in \Pi(p,q)} \mathbb{E}_{(x,y)\sim\gamma} \|x - y\|$$

Why it fixes saturation: the Wasserstein distance has a meaningful gradient even when the two distributions have no overlap. The critic (D in WGAN) always provides a useful learning signal, even at the very start of training when the generator is terrible (Arjovsky et al., 2017, ICML).

Fix 2 — Gradient Penalty (WGAN-GP)

The Lipschitz constraint: Wasserstein distance requires the critic D to be "1-Lipschitz": its output cannot change faster than 1 unit per unit of input change. Think of it as requiring the critic to be *consistent* — similar inputs get similar scores.

The penalty: instead of hard constraints (weight clipping, which causes gradient explosion), Gulrajani et al. (2017, NeurIPS)[3] add a soft penalty to the training loss:

$$\mathcal{L}_{\text{GP}} = \lambda \cdot \mathbb{E}_{\hat{x}} \left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2 \right]$$

where $\hat{x}$ is a random mixture of real and fake. If the gradient norm deviates from 1, the critic is penalised. **Fixed values:** $\lambda = 10$, $n_{\text{critic}} = 5$ (the critic is updated 5 times per generator step). These are not hyperparameters to search over — they are literature-anchored constants.

What is a Neural Network? (Calculus 1 Perspective)

In one sentence: a neural network is a very complicated function with millions of adjustable parameters, organised in layers, that can approximate any pattern if given enough parameters and training examples.

Calculus 1 version: you already know how to minimise $f(x) = x^2$: take the derivative $f'(x) = 2x$, set it to zero. A neural network minimises a loss function $\mathcal{L}(\theta)$ over millions of parameters θ using the same idea: repeatedly subtract a small multiple of the gradient $\nabla_\theta \mathcal{L}$:

$$\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$$

This is called **gradient descent**. The gradient $\nabla_\theta \mathcal{L}$ is computed by the chain rule (Calculus 1: derivative of a composed function). Called **backpropagation** in neural-network literature.

The Vanishing Gradient Problem

In one sentence: in a deep network, the gradient signal shrinks exponentially as it travels backward through layers, leaving early layers unable to learn.

The chain rule: for a 50-layer network, the chain rule gives:

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial h_{50}} \cdot \frac{\partial h_{50}}{\partial h_{49}} \cdots \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1}$$

Each $\frac{\partial h_{i+1}}{\partial h_i} \approx 0.5$ in a typical network. After 50 layers: $0.5^{50} \approx 10^{-15}$. Essentially zero. Layer 1 gets no useful gradient.

Relevance here: (1) GRU cells solve this for the time dimension via gating. (2) WGAN-GP provides well-behaved gradients to the generator even when distributions are far apart, avoiding the saturation-to-zero problem of the standard GAN loss.

Adam: The Smart Optimizer

In one sentence: Adam adjusts the step size differently for each parameter, taking small steps in directions you've been inconsistent and large steps in directions you've been consistent.

Hiking analogy: imagine hiking in fog. Where you've stumbled a lot (high past gradient variance), take small careful steps. Where you've been walking steadily in one direction, stride longer. Adam keeps track of both the average gradient (momentum, β_1) and the average squared gradient (variance, β_2) per parameter.

$\beta_1 = 0$ for WGAN-GP: In adversarial training, the correct direction for the generator changes every step (because the critic updates between each generator step). Momentum would carry old, now-wrong information. Setting $\beta_1 = 0$ uses only the *current* gradient. $\beta_2 = 0.9$ is standard for all models (Gulrajani et al., 2017).

seq_len: What Counts as One Training Example?

In one sentence: the model processes the time series as fixed-length windows of `seq_len=128` consecutive days; each window is one training example.

Why 128? Long enough to capture volatility clustering (which persists for days to weeks), short enough to create thousands of overlapping training examples from a 5-year dataset.

Sliding window: from a 1 000-day series we get $1\,000 - 128 + 1 = 873$ windows. Each market's 80 % training split (≈800 days) yields ≈673 windows — enough to learn distributional patterns. Models train on one market's windows only; no cross-market data is mixed.

FinGAN constraint: `seq_len` must be divisible by 8. Three `ConvTranspose1d` layers each double the length ($2^3 = 8$); FinGAN starts from noise of length `seq_len`/8 and expands to `seq_len`.

GRU: Memory Cells for Sequential Data (TimeGAN)

In one sentence: a GRU (Gated Recurrent Unit, Cho et al. 2014)[19] is a mathematical "memory" that reads a sequence one step at a time and decides at each step what to remember and what to forget.

The gate analogy: imagine reading market news one headline at a time. At each new headline you make two decisions: (1) **Update gate** z_t: is this headline important enough to UPDATE my view? (2) **Reset gate** r_t: should I FORGET my past view before reading this?

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t, \quad z_t = \sigma(W_z[h_{t-1}, x_t])$$

h_t is the GRU's "memory" (hidden state). The sigmoid σ makes the gate a smooth 0-to-1 switch.

Why GRU in TimeGAN: financial volatility clustering spans days to weeks. GRU maintains context across these time scales without vanishing gradients (the gates prevent gradient shrinkage). Yoon et al. (2019)[4] validated GRU as the backbone for sequential GAN training at NeurIPS.

Conv1d + TCN: Pattern Detection at Multiple Scales (QuantGAN)

Conv1d in one sentence: slide a small learned "pattern template" (kernel) along the time series; the output is a score for how well the template matches each position.

Analogy: a spell-checker slides a window of 5 characters across a document, checking for known misspelling patterns. Similarly, a Conv1d kernel of length 5 slides over 128 days of returns, scoring each 5-day window on a learned pattern (e.g., reversal, momentum spike).

TCN (Temporal Convolutional Network): stacks Conv1d layers but with *exponentially growing gaps* (dilations). Layer 1 looks at today ±1 day. Layer 2 at today ±2 days. Layer 3 at today ±4 days. After 7 layers: receptive field covers 127 days — the entire window — with only 7 layers worth of parameters (Bai et al., 2018[20], arXiv[20]).

Why QuantGAN uses TCN: multi-scale financial patterns (intraday microstructure, weekly seasonality, monthly cycles) require exactly this hierarchical temporal coverage. Wiese et al. (2020, *Quantitative Finance*)[5] showed TCN+WGAN-GP outperforms RNN-based generators on Hurst exponent fidelity.

ConvTranspose1d: The Expanding Generator (FinGAN)

In one sentence: ConvTranspose1d is the *reverse* of Conv1d — instead of compressing a long signal into fewer values, it *expands* a short vector into a longer sequence.

Video upscaling analogy: a 32-pixel image scaled to 96 pixels — each output pixel is a weighted sum of nearby input pixels, but the weights "spread" information outward. ConvTranspose1d does the same in 1D.

FinGAN's Generator: takes noise z of length 16 → ConvTranspose1d → length 32 → ConvTranspose1d → length 64 → ConvTranspose1d → length 128 (= `seq_len`). Each step doubles the length and halves the "abstraction". The model learns to produce realistic return sequences from random seeds.

LayerNorm: Keeping Values Calibrated

In one sentence: for each sample *independently*, subtract its mean and divide by its standard deviation before passing to the next layer — keeps activations in a stable numerical range.

Calibration analogy: grading multiple exams, some scored 0–10, others 0–100. Before comparing, rescale each exam to a 0–1 range. LayerNorm does this for each neural network "sample" individually.

Why not BatchNorm? BatchNorm normalises *across* the batch (all samples together). With small batches (32–64 samples), batch statistics are noisy, causing unstable GAN training. LayerNorm normalises *within* each sample, independent of batch size.

In FinGAN's Critic: the critic receives both real returns (say, scale 0.01–0.03 std) and early-training fakes (scale 0.5–5.0 std). LayerNorm prevents the critic from using scale alone to distinguish them — forcing it to learn distributional *shape* differences instead.

Knowledge Base III — TimeGAN, QuantGAN, and FinGAN Side by Side

What each architecture assumes about financial time series, and why

	TimeGAN	QuantGAN	FinGAN
Core architecture	4-phase (encoder + supervisor + GAN) using GRU	TCN-based WGAN-GP	CNN deconvolution WGAN-GP
Temporal mechanism	Recurrent (GRU): processes each step in order	Dilated convolutions: parallel look at all scales	Transposed convolutions: upsamples from short to long
Training method	BCE + moment-matching loss; 4 separate training phases	WGAN-GP (single phase)	WGAN-GP (single phase)
Literature reference	Yoon et al. (2019, NeurIPS)[4]	Wiese et al. (2020, *Quant. Finance*)[5]	Custom (this paper)
Key hyperparameters	hidden_dim, num_layers, lr	lr, noise_dim (n_critic=5 fixed)	lr, noise_dim (n_critic=5 fixed)
seq_len constraint	Any length	Any length	Must be divisible by 8
Inductive bias	Temporal ordering matters step-by-step	Multi-scale patterns (short+long memory)	Generate from compressed noise, like image GANs
BRICS hypothesis	GRU memory captures volatility regime persistence	Dilations capture BRICS multi-scale clustering	Deconv captures heavy-tail shape from noise

TimeGAN: Four Phases to Learn Temporal Structure

Yoon et al. (2019)[4] recognised that a plain GAN ignores the *order* of time steps. Their solution: embed data into a learned latent space first, then generate in that space.

Phase 1 — Autoencoder: train an Encoder (E) and Recovery (R) so that $R(E(x)) \approx x$. This builds a compressed "language" the GAN will speak. Like learning music notation before composing.

Phase 2 — Supervisor: train a Supervisor (S) that predicts the *next* latent step h_{t+1} from the current one h_t. This injects temporal causality into the model.

Phase 3 — Joint GAN: train G and D simultaneously in the latent space, with the supervisor providing an extra loss term $\mathcal{L}_S = \|h_{t+1} - S(h_t)\|^2$ (moment matching).

Phase 4 — Refinement: fine-tune the recovery R with generated latent sequences. Final output: $\tilde{x} = R(G(z))$.

Limitation here: `epochs=5` means each phase gets ∼2 gradient steps. TimeGAN's GRU cannot converge in 2 steps (Yoon et al. used 5 000–10 000 iterations per phase). **This is the most critical open bug.**

QuantGAN: Dilated Patterns Across Time Scales

Wiese et al. (2020)[5] adapted the WGAN-GP framework to financial time series by replacing the usual MLP with a Temporal Convolutional Network (TCN).

Generator G: noise z of length `seq_len` → stack of dilated Conv1d layers → synthetic returns. Each layer's dilation doubles ($d = 1, 2, 4, 8, \ldots$), so layer k sees the entire context with only 2^k effective lags.

Critic D: same TCN structure, but maps sequences to a single scalar ("how real is this?"). WGAN-GP training: critic updated $n_{\text{critic}} = 5$ times per generator step.

Why this works for BRICS: volatility clustering in BRICS markets operates simultaneously at intraday, weekly, and monthly time scales. Dilated convolutions cover all three scales in a single forward pass. Wiese et al. showed this architecture reproduces the Hurst exponent of equity returns significantly better than LSTM-based generators.

FinGAN: Generating Sequences like Images

FinGAN borrows from image generation (Radford et al., DCGAN 2015)[21] and applies it to 1-D financial sequences.

Generator G: noise z (dimension 100) $\xrightarrow{\text{FC}}$ tensor of shape (64, 16) $\xrightarrow{\text{ConvT1d}\times 3}$ (1, 128). Each ConvTranspose1d doubles the temporal length ($16 \to 32 \to 64 \to 128$) while reducing channels ($64 \to 32 \to 16 \to 1$).

Critic D: mirrors G in reverse. Sequence (1, 128) $\xrightarrow{\text{Conv1d}\times 3}$ features (64, 16) $\to$ scalar. LayerNorm after each Conv1d stabilises training.

Inductive bias: deconvolution builds long sequences from hierarchical abstractions, the same way an image GAN builds textures from abstract "concepts". For finance, this means the model first decides the broad return regime (high/low volatility) and then fills in fine-grained daily fluctuations.

Novelty: no prior paper applies CNN deconvolution WGAN-GP to BRICS markets with explicit stylized-fact evaluation. This is a direct contribution of the current study.

Why Compare All Three? The Inductive Bias Argument

Each architecture encodes a different prior belief about how financial time series work:

- **TimeGAN (GRU)**: "returns are causally ordered; today's volatility is a function of recent history." This is the textbook financial econometrics view.
- **QuantGAN (TCN)**: "volatility operates at multiple time scales simultaneously." This aligns with multi-fractal market models.
- **FinGAN (CNN)**: "market regimes are hierarchical; fine-grained returns are conditioned on a macro-state." This aligns with hidden Markov / regime-switching models.

Comparing all three reveals *which structural assumption best describes BRICS markets* — a theoretically interesting result, not just an engineering comparison. This is the core scientific question the paper answers.

ACF / Autocorrelation Function

In one sentence: measures how similar a time series is to a shifted copy of itself.
$\rho_k = \text{Corr}(r_t, r_{t-k})$. For log returns: $\rho_k \approx 0$ (efficient markets). For $|r_t|$: $\rho_k > 0$ for $k = 1$–$100+$ (volatility clustering). A good GAN reproduces both patterns simultaneously. Our metric: mean absolute error between the real and synthetic ACF curves (ACF MAE).

ARCH Effects / ARCH-LM Test

In one sentence: ARCH effects mean "the variance of returns today depends on the variance yesterday" — volatility comes in clusters.
Named after Robert Engle (1982)[10][10] Nobel Prize). The LM test regresses $\hat{r}_t^2$ on its own lags; a significant χ^2 statistic means ARCH is present. BRICS returns almost always show strong ARCH. A synthetic series that does NOT show ARCH has failed to capture a key stylized fact.

Discriminative AUC

In one sentence: train a simple classifier to tell real from fake; if it can't do better than coin-flipping (AUC = 0.5), the synthetic data is indistinguishable.
AUC (Area Under the ROC Curve) = probability that the classifier ranks a real sample higher than a fake sample. AUC = 1.0 means trivially separable; AUC = 0.5 means perfect indistinguishability. Our implementation: logistic regression on 20-day rolling-window features (mean, std, mean-abs, mean-sq), 5-fold cross-validation. Based on TSGBench (Ang et al., 2023[15], PVLDB[15])[15].

Energy Distance

In one sentence: a proper mathematical distance between two distributions, computed from pairwise distances between samples — robust to serial dependence.
$E(P,Q) = 2\mathbb{E}\|X-Y\| - \mathbb{E}\|X-X'\| - \mathbb{E}\|Y-Y'\|$, where $X, X' \sim P$ (real), $Y, Y' \sim Q$ (synthetic). Székely & Rizzo (2004). $E \geq 0$; $E = 0 \Leftrightarrow P = Q$. Unlike KS p-values, the energy distance is interpretable even with serially correlated data, because it doesn't assume i.i.d. samples.

Hurst Exponent (H)

In one sentence: measures how "trendy" a time series is; $H > 0.5$ means past moves predict future moves.
Estimated via the R/S (Rescaled Range) method applied to *absolute returns* $|r_t|$: $\mathbb{E}[R_n/S_n] \sim c \cdot n^H$. For $H = 0.5$: random walk (Brownian motion). For $H > 0.5$: long memory — large $|r|$ today predicts large $|r|$ tomorrow. Typical daily $|r_t|$: $H \approx 0.6$–0.7 (Ding et al., 1993[8]).
Debate: rough-volatility theory (Gatheral et al., 2018) estimates $H \approx 0.1$ on intraday *realised variance* using a different estimator at a different time scale — both camps can be correct simultaneously (Cont & Das, 2024[18]). Our metric: $|H_{\text{real}} - H_{\text{syn}}|$ on daily $|r_t|$ via R/S.

KS Test (Kolmogorov-Smirnov)

In one sentence: tests whether two samples could plausibly come from the same distribution, using the maximum gap between their empirical CDFs.
$D = \max_x |F_n(x) - G_m(x)|$. Under H_0 (same distribution), the p-value should be uniform on [0,1]. **Limitation:** assumes i.i.d. samples. Financial returns are serially correlated ($\rho_k \neq 0$ for $|r_t|$), inflating the test statistic and making p-values anti-conservative ($n_{\text{eff}} = n/11$ to $n/31$). Retained in the report with caveat; not used as the primary metric.

Quantile MSE (QMSE)

In one sentence: compare the k-th smallest real return to the k-th smallest synthetic return, for $k = 1\%$ to 99% of the sorted series.

$$\text{QMSE} = \frac{1}{K}\sum_{k=1}^{K}\left(Q_{\text{real}}(\alpha_k) - Q_{\text{syn}}(\alpha_k)\right)^2$$

This is the only sensible "pointwise" comparison between two unaligned distributions — comparing by rank, not by date. A large QMSE at the tails (1%, 5%, 95%, 99%) means the GAN misses extreme events. Equivalent to the L^2 distance between empirical CDFs.

Tail Index (Hill Estimator)

In one sentence: measures how "extreme" the extreme events are; lower tail index = heavier tail = more frequent crashes/spikes.
The Hill estimator: sort $|r_t|$ in ascending order; take the largest k values; $\hat{\alpha} = k/\sum \log(|r|_{(n-i)}/|r|_{(n-k)})$. For a normal distribution: $\alpha = \infty$ (no power-law tail). For financial returns: $\alpha \approx 2$–4 (Pareto-like tails). Lower α in BRICS vs developed markets is why BRICS are a more interesting test case.

Volatility Clustering

In one sentence: large price moves tend to be followed by large moves (regardless of direction), and small moves by small moves.
Mathematically: $|\text{Corr}(|r_t|, |r_{t+k}|)| > 0$ for $k = 1, 2, \ldots, 100$. This is NOT saying direction repeats (that would violate market efficiency); only that *magnitude* repeats. Mandelbrot (1963)[9] observed this first. Our metric: ACF MAE on $|r_t|$ and r_t^2.

Walk-Forward Validation

In one sentence: instead of one train/test split, roll the training window forward 5 times and report the mean and standard deviation of results across folds.
Why: a single train/test split gives exactly one metric score. Is this score lucky or representative? Walk-forward gives 5 scores, enabling mean $\pm$ std — the standard robustness evidence required for time-series papers (Tashman 2000[13]; Bergmeir & Benítez 2012[14]). Our implementation: 5 folds per market, model retrained from scratch each fold on that market's own test series — no cross-market mixing.

Wasserstein Distance

In one sentence: the minimum work needed to rearrange the fake distribution into the real one, where "work" = mass $\times$ distance moved.
$W_1(p, q) = \inf_\gamma \mathbb{E}_{(x,y)\sim\gamma}\|x - y\|$ where γ is any joint distribution with marginals p and q. The Kantorovich-Rubinstein duality makes this tractable: $W_1(p, q) = \sup_{\|f\|_L \leq 1}(\mathbb{E}_p[f] - \mathbb{E}_q[f])$. WGAN-GP trains the critic f to be the optimal 1-Lipschitz function. Our metric: empirical Wasserstein-1 (implemented by `scipy.stats.wasserstein_distance`).

The 10 Stylized Facts in Plain Language

1. **Fat tails:** extreme returns happen far more often than Gaussian.
2. **Near-zero ACF:** today's return direction doesn't predict tomorrow's direction.
3. **Volatility clustering:** today's *magnitude does* predict tomorrow's magnitude.
4. **Long memory:** the ACF of $|r_t|$ decays slowly (weeks to months).
5. **Negative skewness:** crashes are faster and sharper than rallies.
6. **Leverage effect:** drops in price $\rightarrow$ increases in volatility (asymmetric).
7. **Gain/loss asymmetry:** losses cluster more tightly than gains.
8. **ARCH effects:** conditional variance is predictable (Engle, 1982[10]).
9. **Extreme events:** returns beyond 2σ occur >5 % of days (vs 4.6 % Gaussian).
10. **Distributional match:** full CDF shape, not just first 4 moments.

A GAN that reproduces all 10 is considered finance-grade. None of the three tested models perfectly reproduces all 10 on BRICS data.

References

[1] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative Adversarial Nets. *Advances in Neural Information Processing Systems (NeurIPS)*, 27, 2672–2680.

[2] Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein Generative Adversarial Networks. *International Conference on Machine Learning (ICML)*, Proceedings of Machine Learning Research 70, pp. 214–223.

[3] Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. (2017). Improved Training of Wasserstein GANs. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.

[4] Yoon, J., Jarrett, D., & van der Schaar, M. (2019). Time-series Generative Adversarial Networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 32, 5508–5518.

[5] Wiese, M., Knobloch, R., Korn, R., & Kretschmer, P. (2020). Quant GANs: Deep Generation of Financial Time Series. *Quantitative Finance*, 20(9), 1419–1440.

[6] Adams, Z., Füß, R., & Glück, T. (2019). Are correlations constant? Empirical and theoretical results on popular distributional assumptions in finance. *Financial Management*, 48(3), 793–826.

[7] Cont, R. (2001). Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2), 223–236.

[8] Ding, Z., Granger, C. W. J., & Engle, R. F. (1993). A long memory property of stock market returns and a new model. *Journal of Empirical Finance*, 1(1), 83–106.

[9] Mandelbrot, B. (1963). The variation of certain speculative prices. *The Journal of Business*, 36(4), 394–419.

[10] Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007.

[11] Székely, G. J., & Rizzo, M. L. (2004). Testing for equal distributions in high dimension. *InterStat*, 5(16.10), 1249–1272.

[12] Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research (JMLR)*, 13, 281–305.

[13] Tashman, L. J. (2000). Out-of-sample tests of forecasting accuracy: an analysis and review. *International Journal of Forecasting*, 16(4), 437–450.

[14] Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191, 192–213.

[15] Ang, Y., Bansal, P., Gupta, G., & Su, L. (2023). TSGBench: Time Series Generation Benchmark. *Proceedings of the VLDB Endowment (PVLDB)*, 17(3), 305–318.

[16] Liao, Q. et al. (2025). CTBench: A Comprehensive and Extensible Benchmark for Conditional Time Series Generation. *Advances in Neural Information Processing Systems (NeurIPS)*, 38, Datasets and Benchmarks Track.

[17] Gatheral, J., Jaisson, T., & Rosenbaum, M. (2018). Volatility is rough. *Quantitative Finance*, 18(6), 933–949.

[18] Cont, R., & Das, S. (2024). Rough volatility: fact or artefact? *Sankhya B*, 86(1), 191–223.

[19] Cho, K., van Merrienboer, B., Gülçehre, Ç., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *Proceedings of EMNLP 2014*, pp. 1724–1734.

[20] Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv:1803.01271*.

[21] Radford, A., Metz, L., & Chintala, S. (2015). Unsupervised representation learning with deep convolutional generative adversarial networks (DCGAN). *arXiv:1511.06434*. Published at ICLR 2016.

[22] Lobato, I. N., & Savin, N. E. (1998). Real and spurious long-memory properties of stock-market data. *Journal of Business & Economic Statistics*, 16(3), 261–268.

[23] Hurst, H. E. (1951). Long-term storage capacity of reservoirs. *Transactions of the American Society of Civil Engineers*, 116, 770–799.

[24] Mandelbrot, B., & Wallis, J. R. (1969). Robustness of the rescaled range R/S in the measurement of noncyclic long-run statistical dependence. *Water Resources Research*, 5(5), 967–988.

[25] Ma, J., Chen, Z., Ding, M., Tang, J., & Zheng, Y. (2024). GenTS: Generative Time Series via Large Language Models. *arXiv preprint*.

[26] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD 2019)*, pp. 2623–2631.

[27] Meldrum, A. et al. (2025). Deep generative models for financial time series: a comparative study. *arXiv:2510.26076*.

[28] SFAG Team. (2026). Synthetic Financial Asset Generator (SFAG): benchmarking GAN-based models on emerging-market log-returns. *arXiv:2601.12990*.

[29] Takahashi, S., & Mizuno, T. (2025). Diffusion-based generation of financial time series with stylized-fact constraints. *Quantitative Finance*, 25(10).

[30] Gruver, N., Finzi, M., Qiu, S., & Wilson, A. G. (2023). Large language models are zero-shot time series forecasters. *Advances in Neural Information Processing Systems (NeurIPS)*, 36.

[31] Ansari, A. F., Stella, L., Turkmen, C., Zhang, X., Mergenthaler, P., Shchur, O., Rangapuram, S. S., Pineda Arango, S., Kapoor, S., Zschiegner, J., Maddix, D. C., Wang, H., Mahoney, M. W., Januschowski, T., de Salvo, B., Gasthaus, J., & Wang, Y. (2024). Chronos: Learning the language of time series. *arXiv:2403.07815*.

[32] Hamdouche, M. et al. (2025). Forging time series: a comprehensive study of GAN-based synthetic financial data. *arXiv:2505.17103*.

[33] LeBaron, B. (2001). Stochastic volatility as a simple generator of apparent financial power laws and long memory. *Quantitative Finance*, 1(6), 621–631.